

Java + Spring Boot Developer Intern Interview Notes

Detailed preparation guide: Java basics, OOP, Java 8, Java 9–21, concurrency, Spring Boot, Security, Hibernate/JPA, REST APIs, exception handling, microservices, RabbitMQ, and SQL.

Target: Fresher / Developer Intern / Junior Java Backend interviews

How to use this PDF: Learn the concept → type the example yourself → explain it aloud → solve 2–3 variations → build one small Spring Boot CRUD project.

Interview-first notes • Java 8 through Java 21 • Practical examples

Contents

1. Java Basics
2. OOP Concepts
3. Collections & Generics
4. Java 8 Features
5. Java 9-21: What Changed
6. Exception Handling
7. Concurrency & Deadlock
8. Spring & Spring Boot Basics
9. REST APIs
10. Global Exception Handling
11. JPA & Hibernate
12. Hibernate Relationships
13. Spring Security & JWT
14. Microservices Theory
15. Microservices Patterns
16. RabbitMQ
17. SQL: Basics to Advanced
18. Interview Questions & Rapid Revision
19. Practical Project Blueprint

Note: Java version details are aligned with Oracle's Java language-change documentation. Java 9 introduced modules; Java 10 var; Java 14 switch expressions; Java 15 text blocks; Java 16 records and pattern matching for instanceof; Java 17 sealed classes; Java 21 record patterns and pattern matching for switch.

1. Java Basics

JDK vs JRE vs JVM

- JVM: Runs Java bytecode. It provides the runtime environment and manages memory, threads, class loading and execution.
- JRE: JVM + runtime libraries needed to run Java applications. In modern Java distributions, the separate JRE packaging is less central than it was historically.
- JDK: Development kit containing tools such as the compiler plus the runtime needed to develop and run Java applications.

Compilation flow

```
Source code (.java)
↓ javac
Bytecode (.class)
↓ JVM
Machine/native execution
```

Primitive vs reference types

Primitive	Reference
byte, short, int, long	Objects / classes
float, double	Arrays
char, boolean	String
Stores a value	Stores a reference to an object

String fundamentals

- String is immutable: operations produce a new String rather than changing the existing object.
- Use StringBuilder for repeated modifications, especially inside loops.

```
String s = "Java";
String t = s + " Backend";    // new String result
StringBuilder sb = new StringBuilder("Java");
sb.append(" Backend");
System.out.println(sb);
```

Important keywords

Keyword	Interview meaning
static	Belongs to the class rather than an instance.
final	Variable cannot be reassigned; method cannot be overridden; class cannot be extended.
this	Current object reference.
super	Parent-class members / constructor access.
new	Creates an object.

Common questions

- Why is Java platform independent?
- What is bytecode?
- Why is String immutable?
- String vs StringBuilder vs StringBuffer?

- What is the difference between == and equals()?

2. OOP Concepts

Object-oriented programming organizes software around objects that combine state and behavior.

Encapsulation

Hide internal state and expose controlled operations. Usually implemented using private fields and public methods.

```
class Student {  
    private int age;  
  
    public void setAge(int age) {  
        if (age >= 0) this.age = age;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

Inheritance

```
class Animal {  
    void eat() { System.out.println("Eating"); }  
}  
class Dog extends Animal {  
    void bark() { System.out.println("Barking"); }  
}
```

Polymorphism

- Overloading: same method name, different parameter list; resolved at compile time.
- Overriding: child class provides a new implementation; resolved at runtime through dynamic dispatch.

```
class Printer {  
    void print(int x) {}  
    void print(String s) {}  
}
```

```
Animal a = new Dog();  
a.eat(); // runtime object is Dog
```

Abstraction

- Expose what an object does while hiding implementation details.

```
interface Payment {  
    void pay(double amount);  
}
```

```
class CardPayment implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid: " + amount);  
    }  
}
```

Abstract class vs interface

Abstract class	Interface
Can have instance state	Primarily defines a contract; can also have default/static methods.
Can have constructors	No instance constructors.

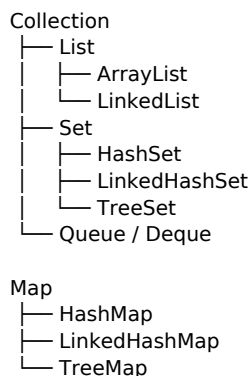
Abstract class	Interface
A class extends one class	A class can implement multiple interfaces.
Good for shared base behavior	Good for capabilities/contracts.

Other OOP concepts to know

- Composition: a class contains another object (HAS-A). Often preferred over deep inheritance.
- Association: objects know/use each other.
- Aggregation: weaker whole-part relationship.
- Coupling: degree of dependency between components; lower coupling generally improves maintainability.
- Cohesion: how closely related responsibilities within a component are; higher cohesion is desirable.

3. Collections & Generics

Core hierarchy



Type	Main property	Typical use
ArrayList	Fast random access; dynamic array	General-purpose list
LinkedList	Linked nodes; efficient end operations	Specialized deque/list cases
HashSet	Unique elements; no ordering guarantee	Fast membership / deduplication
LinkedHashSet	Unique + insertion order	Dedup while retaining order
TreeSet	Sorted unique elements	Sorted set
HashMap	Key-value; no ordering guarantee	Lookup by key
LinkedHashMap	Key-value + predictable insertion/access order	Ordered map / cache patterns
TreeMap	Sorted by key	Navigable sorted map

HashMap interview model

- A hash function helps select a bucket from a key's hash.
- The map compares keys using hash-related checks and equals() to identify the matching key.
- Average-case lookup/insert/delete are generally near O(1), but exact behavior depends on collisions and implementation details.

Comparable vs Comparator

```
// Comparable: natural ordering
class Student implements Comparable<Student> {
```

```

    int age;
    public int compareTo(Student other) {
        return Integer.compare(this.age, other.age);
    }
}

// Comparator: external/custom ordering
students.sort(Comparator.comparingInt(s -> s.age));

```

Generics

```

List<String> names = new ArrayList<>();
names.add("Asha");
// names.add(10); // compile-time error

```

4. Java 8 Features

Java 8 is a major interview milestone because it introduced functional programming features that remain common in production Java.

Lambda expressions

```

// Traditional
Runnable r1 = new Runnable() {
    public void run() {
        System.out.println("Hello");
    }
};

// Lambda
Runnable r2 = () -> System.out.println("Hello");

```

Functional interface

- An interface intended to have one abstract method. @FunctionalInterface lets the compiler verify that intent.

```

@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}

```

```

Calculator c = (a, b) -> a + b;
System.out.println(c.add(2, 3));

```

Method references

```

List<String> names = Arrays.asList("A", "B");
names.forEach(System.out::println);

```

Streams

- A stream represents a pipeline for processing data. Intermediate operations are generally lazy; terminal operations trigger execution.

```

List<Integer> nums =
    Arrays.asList(1,2,3,4,5,6);

List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 10)
    .collect(Collectors.toList());

```

Operation	Purpose
filter	Keep elements matching a predicate
map	Transform each element

Operation	Purpose
distinct	Remove duplicates
sorted	Sort elements
limit	Take at most N elements
skip	Ignore first N elements
count	Count elements
reduce	Combine elements into one result
collect	Materialize into a collection/result
forEach	Consume each element

Optional

```
Optional<String> name = Optional.ofNullable(input);
```

```
String result = name
    .map(String::trim)
    .filter(s -> !s.isEmpty())
    .orElse("Unknown");
```

Date and Time API

```
LocalDate today = LocalDate.now();
LocalDate nextWeek = today.plusWeeks(1);
Duration d = Duration.ofHours(2);
```

Interview focus

- Lambda vs anonymous class
- map vs filter
- map vs flatMap
- Stream vs Collection
- Why are intermediate stream operations lazy?
- Optional: when is it useful and when should it not be abused?

5. Java 9-21: What Changed

For interviews, learn the high-value language milestones rather than memorizing every JDK API change. Oracle's Java 21 language-change summary identifies the major permanent language features across this range. [cite](#) [turn0search0](#) [turn0search1](#)

Version	Key feature(s)	What to remember
9	Module System	Named modules, requires/exports, stronger encapsulation.
10	var	Local variable type inference; compiler still determines a static type.
11	Lambda parameter syntax	Allows var in lambda parameters in applicable cases.
12-13	Switch expression previews	Evolution toward expression-style switch.
14	Switch expressions	switch can produce a value; arrow cases reduce fall-through.
15	Text blocks	Readable multiline string literals.
16	Records; pattern matching for instanceof	Compact data carriers and less casting boilerplate.

Version	Key feature(s)	What to remember
17	Sealed classes	Restrict which classes/interfaces can extend/implement a type.
18	UTF-8 default	UTF-8 is the default charset for standard Java APIs where specified.
19-20	Preview evolution	Virtual threads and pattern features progressed through preview stages.
21	Record patterns; pattern matching; major switch improvements	Major switch improvements, LTS.

var

```
var name = "Addy";    // inferred as String
var numbers = new ArrayList<Integer>();
```

Switch expression

```
int days = switch (month) {
    case 1, 3, 5, 7, 8, 10, 12 -> 31;
    case 4, 6, 9, 11 -> 30;
    case 2 -> 28;
    default -> throw new IllegalArgumentException();
};
```

Text blocks

```
String json = """
{
    "name": "Java",
    "version": 21
}
""";
```

Record

```
record Student(int id, String name) {}
```

```
Student s = new Student(1, "Asha");
System.out.println(s.name());
```

- Records are concise data carriers. They automatically provide accessors, equals, hashCode and toString based on their components.

Sealed class

```
sealed interface Shape permits Circle, Rectangle {}
```

```
final class Circle implements Shape {}
final class Rectangle implements Shape {}
```

Pattern matching for instanceof

```
if (obj instanceof String s && !s.isBlank()) {
    System.out.println(s.length());
}
```

Pattern matching for switch

```
String result = switch (obj) {
    case String s -> "String: " + s;
    case Integer i -> "Integer: " + i;
    case null -> "null";
    default -> "Other";
};
```

Virtual threads - concept

- Java 21 made virtual threads permanent. They are lightweight threads designed for high-concurrency workloads, especially blocking I/O.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> callRemoteService());
}
```

Interview answer

If asked 'What are the important Java 9–21 features?', prioritize: modules, var, switch expressions, text blocks, records, pattern matching, sealed classes, and Java 21 virtual threads. Do not spend most of your preparation memorizing obscure release notes.

6. Exception Handling

Hierarchy

```
Throwable
├── Error
├── Exception
│   ├── RuntimeException (unchecked)
│   └── checked exceptions
```

- Checked: compiler requires handling or declaration for applicable checked exception types.
- Unchecked: RuntimeException and its subclasses; compiler does not force catch/declare.

try/catch/finally

```
try {
    int x = 10 / value;
} catch (ArithmeticException e) {
    System.out.println("Invalid division");
} finally {
    System.out.println("Cleanup");
}
```

throw vs throws

- throw: explicitly creates/throws an exception instance.
- throws: declares that a method may propagate specified exceptions.

```
void readFile() throws IOException {
    throw new IOException("Read failed");
}
```

Custom exception

```
class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}
```

Best practices

- Catch the most specific exception you can meaningfully handle.
- Do not silently swallow exceptions.
- Use meaningful messages and preserve the cause when wrapping exceptions.
- Do not use exceptions for ordinary control flow.
- Use try-with-resources for Closeable/AutoCloseable resources.

7. Concurrency & Deadlock

Deadlock

A deadlock occurs when threads wait indefinitely for locks held by one another.

Thread A: holds Lock 1 → waits for Lock 2
Thread B: holds Lock 2 → waits for Lock 1

Four classic conditions

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

Prevention

- Acquire multiple locks in a consistent global order.
- Keep synchronized sections small.
- Avoid unnecessary shared mutable state.
- Use timed lock acquisition where appropriate (e.g., `ReentrantLock.tryLock()`).

Other concurrency concepts

- Race condition: result depends on timing/interleaving of threads.
- synchronized: provides mutual exclusion and memory-visibility guarantees.
- volatile: guarantees visibility/order properties for a variable but does not make compound operations atomic.
- `ExecutorService`: manages task execution using thread pools/executors.
- Atomic classes: support lock-free atomic operations for common cases.

8. Spring & Spring Boot Basics

Spring core idea

- IoC (Inversion of Control): the framework manages object creation/wiring rather than every class constructing its dependencies itself.
- Dependency Injection: dependencies are supplied to a class, improving testability and reducing coupling.

Common stereotypes

Annotation	Typical role
<code>@Component</code>	Generic Spring-managed component
<code>@Service</code>	Service/business layer
<code>@Repository</code>	Persistence/data-access layer
<code>@Controller</code>	MVC controller
<code>@RestController</code>	Controller whose methods commonly return response bodies
<code>@Configuration</code>	Java configuration class
<code>@Bean</code>	Explicit bean definition method

Constructor injection

```
@Service
public class UserService {
    private final UserRepository repo;

    public UserService(UserRepository repo) {
```

```

        this.repo = repo;
    }
}

```

Typical layered architecture

```

Client
↓
Controller → HTTP / JSON
↓
Service → business rules
↓
Repository → persistence
↓
Database

```

Spring Boot

- Provides conventions, auto-configuration, starters and embedded server support that reduce setup work.
- Configuration is commonly placed in application.properties or application.yml.

Common interview questions

- What is dependency injection?
- What is a Spring bean?
- Why use constructor injection?
- @Component vs @Service vs @Repository?
- What does @SpringBootApplication represent conceptually?

9. REST APIs

REST is an architectural style for resource-oriented network APIs. In a typical Spring Boot service, HTTP methods map to operations on resources.

Method	Typical meaning	Example
GET	Read	GET /users/10
POST	Create / trigger processing	POST /users
PUT	Replace/update a resource	PUT /users/10
PATCH	Partial update	PATCH /users/10
DELETE	Delete	DELETE /users/10

Status codes

200	OK / successful request
201	Created
204	Success with no response body
400	Bad Request
401	Unauthenticated / missing or invalid authentication
403	Authenticated but not allowed
404	Resource not found
409	Conflict

422	Unprocessable content (commonly used by APIs)
500	Internal server error

Spring REST example

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return service.findById(id);
    }

    @PostMapping
    public ResponseEntity<User> create(@RequestBody User user) {
        User saved = service.create(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

Path variable vs query parameter

- Path variable identifies a resource: /users/10
- Query parameter filters/options: /users?role=admin&page;=2

DTOs

- A DTO (Data Transfer Object) defines the API-facing shape rather than exposing persistence entities directly.
- DTOs help control fields, validation and compatibility between internal models and external API contracts.

10. Global Exception Handling

Instead of putting try/catch blocks in every controller, Spring can centralize HTTP error mapping.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(
        UserNotFoundException ex) {

        Map<String, Object> body = new HashMap<>();
        body.put("message", ex.getMessage());
        body.put("status", 404);

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }
}
```

Recommended error response

```
{
  "timestamp": "2026-09-02T10:15:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "User 10 not found",
  "path": "/users/10"
}
```

Validation

```
public class CreateUserRequest {
    @NotBlank
```

```

private String name;

    @Email
    private String email;
}

```

- Use validation annotations and handle validation failures centrally.
- Do not expose stack traces, SQL details or internal implementation details to clients in production.

11. JPA & Hibernate

JPA vs Hibernate

- JPA (Jakarta Persistence) is a specification/API for object-relational persistence.
- Hibernate ORM is a popular implementation of the JPA specification and also provides additional features.

Entity example

```

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String email;
}

```

Persistence concepts

- Entity: object mapped to a database table.
- Primary key: uniquely identifies an entity.
- Persistence context: set of managed entity instances associated with an EntityManager/session.
- Dirty checking: changes to managed entities can be detected and synchronized to the database at flush/transaction boundaries.
- Lazy loading: related data is loaded when accessed, subject to persistence context/session behavior.
- Eager loading: relation is fetched earlier; can cause unnecessary data loading if used carelessly.

Repository example

```

public interface UserRepository
    extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);
}

```

Common pitfalls

- N+1 query problem: loading a list and then issuing additional queries per entity for relationships.
- Returning entities directly from APIs can expose internal structure and trigger lazy-loading surprises.
- Careless eager relationships can create large joins and memory/performance problems.

12. Hibernate Relationships

Relationship	Example	Typical mapping
One-to-One	Person ↔ Passport	@OneToOne
One-to-Many	Department → Employees	@OneToMany
Many-to-One	Employees → Department	@ManyToOne
Many-to-Many	Students ↔ Courses	@ManyToMany

Many-to-One example

```
@Entity
class Employee {
    @Id
    @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

One-to-Many bidirectional

```
@Entity
class Department {
    @OneToMany(mappedBy = "department")
    private List<Employee> employees = new ArrayList<>();
}

@Entity
class Employee {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

mappedBy

- Indicates the inverse side of a bidirectional relationship; it points to the owning-side field name.

cascade

- Controls which persistence operations propagate from parent to related entities. Do not use CascadeType.ALL blindly.

Many-to-many

- Usually represented using a join table. In real applications, if the relationship has attributes of its own, model the join table as an entity rather than a direct @ManyToMany.

13. Spring Security & JWT

Spring Security provides authentication and authorization mechanisms for applications. When Spring Security is on the classpath, Spring Boot web applications are secured by default; custom configuration is normally required for application-specific rules. [\[cite turn0search9\]](#)

Authentication vs Authorization

- Authentication: Who are you?
- Authorization: What are you allowed to do?

JWT flow

```

Client → POST /login
↓
Authentication
↓
JWT issued
↓
Client stores token
↓
Authorization: Bearer <token>
↓
Security filter validates token
↓
Controller / service

```

SecurityFilterChain concept

```

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    return http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .anyRequest().authenticated()
        )
        .build();
}

```

- Do not disable CSRF automatically in every application; the correct configuration depends on the application architecture (browser session-based vs token-based API, etc.).

Password storage

- Never store plaintext passwords.
- Use a strong password hashing algorithm such as BCrypt/Argon2 through a suitable PasswordEncoder.

JWT interview points

- JWT is signed, not automatically encrypted. Do not put secrets/passwords in its payload.
- Validate signature, expiration and relevant claims.
- Think about token lifetime, refresh strategy and revocation requirements.

14. Microservices Theory

Monolith vs microservices

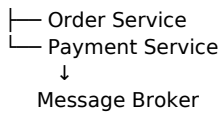
Monolith	Microservices
One deployable application	Multiple independently deployable services
Often simpler initially	Operational complexity is higher
Usually centralized data model	Often service-owned data
Scaling can be coarse-grained	Services can scale independently
In-process calls are simple	Network calls introduce latency/failure modes

Typical architecture

```

Client
↓
API Gateway
├── User Service

```



Why microservices?

- Independent deployment and scaling.
- Team autonomy and clearer service boundaries.
- Technology/data isolation where justified.

Costs

- Distributed tracing/logging/monitoring become important.
- Network failures and latency must be handled.
- Data consistency and distributed transactions become harder.
- Deployment and configuration complexity increases.

Service boundary rule

Do not split an application into microservices just because it is fashionable. A service should have a coherent responsibility and a boundary that can be operated independently.

15. Microservices Patterns

API Gateway

- Single entry point for clients; can handle routing, authentication, rate limiting and aggregation.

Service Discovery

- Services locate instances dynamically rather than hard-coding every address.

Circuit Breaker

- Stops repeated calls to a failing dependency and allows recovery time.

Retry

- Retries transient failures with backoff; never blindly retry every operation, especially non-idempotent writes.

Bulkhead

- Isolates resources so failure/overload in one dependency does not consume everything.

Saga

- Coordinates a business transaction across services using a sequence of local transactions plus compensating actions.

Event-driven architecture

- Services publish events; other services consume them asynchronously.

Idempotency

- Repeated delivery of the same logical request should not create unintended duplicate effects. Important for retries and messaging.

Distributed tracing

- Propagates correlation/trace identifiers across service calls to understand an end-to-end request.

16. RabbitMQ

RabbitMQ is a message broker. Its official tutorials cover queues, work queues, publish/subscribe, routing, topics and RPC, with Java and Spring AMQP examples.

[\[cite\]](#)[\[turn0search7\]](#)[\[turn0search8\]](#)[\[turn0search17\]](#)

Core model

```
graph TD
    Producer --> Exchange
    Exchange -- "(binding + routing key)" --> Queue
    Queue --> Consumer
```

Component	Meaning
Producer	Application that publishes a message.
Exchange	Receives messages and routes them to queues.
Queue	Buffer holding messages for consumers.
Binding	Rule connecting an exchange to a queue.
Routing key	Message attribute used by routing decisions.
Consumer	Application that receives/processes messages.
Acknowledgement	Consumer confirms successful processing.

Exchange types to know

- Direct: route using an exact routing key.
- Fanout: broadcast to bound queues.
- Topic: route using pattern-based routing keys.
- Headers: route based on message headers.

Why messaging?

- Asynchronous processing.
- Decoupling producers and consumers.
- Buffering bursts of work.
- Event-driven integration.

Important reliability ideas

- Acknowledgements: don't acknowledge before the message is safely processed.
- Durability and persistence help messages survive broker restarts, but exact guarantees depend on configuration and failure mode.
- Dead-letter queues can capture messages that cannot be processed successfully.
- Consumers should be designed for idempotency because redelivery can happen.

Spring AMQP mental model

```
@RabbitListener(queues = "orders")
```

```
public void consume(OrderCreatedEvent event) {
    // process event
}
```

17. SQL - Basics to Advanced

Basic CRUD

```
SELECT * FROM employees;
```

```
INSERT INTO employees(name, salary)
VALUES ('Asha', 50000);
```

```
UPDATE employees
SET salary = 55000
WHERE id = 1;
```

```
DELETE FROM employees
WHERE id = 1;
```

Filtering and aggregation

```
SELECT department_id, COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary
FROM employees
WHERE salary >= 30000
GROUP BY department_id
HAVING COUNT(*) >= 2
ORDER BY avg_salary DESC;
```

Joins

```
SELECT e.name, d.name AS department
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.id;
```

Join	Meaning
INNER JOIN	Only matching rows from both sides.
LEFT JOIN	All left rows + matching right rows; unmatched right columns are NULL.
RIGHT JOIN	All right rows + matching left rows.
FULL OUTER JOIN	All rows from both sides; unmatched side is NULL (availability depends on DB).
SELF JOIN	Table joined to itself.

Keys and constraints

- Primary key: uniquely identifies a row.
- Foreign key: references a key in another table.
- UNIQUE: enforces uniqueness.
- NOT NULL: disallows NULL.
- CHECK: enforces a condition where supported.

Normalization

- 1NF: atomic values / no repeating groups.
- 2NF: 1NF + no partial dependency on part of a composite key.
- 3NF: 2NF + no problematic transitive dependency of non-key attributes on other non-key attributes.
- Normalization reduces redundancy; controlled denormalization may be used for performance/reporting.

Indexes

- Indexes speed up suitable reads by providing an alternate access path.
- Indexes consume storage and add write/update overhead.
- Index columns used frequently in selective WHERE, JOIN, ORDER BY or unique constraints, based on actual workload.

Subquery

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

CTE

```
WITH high_paid AS (
    SELECT *
    FROM employees
    WHERE salary > 80000
)
SELECT department_id, COUNT(*)
FROM high_paid
GROUP BY department_id;
```

Window functions

```
SELECT name, department_id, salary,
       DENSE_RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC
       ) AS salary_rank
FROM employees;
```

Transactions and ACID

Property	Meaning
Atomicity	Transaction's changes succeed together or are rolled back together.
Consistency	Transaction moves the database between valid states under its constraints/rules.
Isolation	Concurrent transactions are controlled so their effects meet the chosen isolation semantics.
Durability	Committed changes are preserved according to the database's durability guarantees.

Isolation levels

- Know the concepts of Read Uncommitted, Read Committed, Repeatable Read and Serializable.
- Higher isolation can reduce concurrency/anomalies at the cost of more locking or coordination.

SQL interview must-practice questions

- Second-highest salary.
- Nth highest salary.
- Find duplicate rows.
- Employees with no department.
- Department-wise employee count.
- Top 2 salaries per department using a window function.
- Customers with more than N orders.

18. Interview Questions & Rapid Revision

Q: What is OOP?

Answer: A programming paradigm based on objects that combine state and behavior; key concepts are encapsulation, inheritance, polymorphism and abstraction.

Q: Overloading vs overriding?

Answer: Overloading changes the parameter list; overriding replaces inherited behavior with the same signature in a subclass.

Q: == vs equals()?

Answer: For primitives, == compares values; for references, == compares identity. equals() is intended for logical equality when properly overridden.

Q: ArrayList vs LinkedList?

Answer: ArrayList is usually the default list choice and provides fast indexed access; LinkedList has different trade-offs for linked/deque operations.

Q: HashMap vs HashSet?

Answer: HashMap stores key-value pairs; HashSet stores unique elements and is commonly implemented on top of a hash-based map.

Q: What is a lambda?

Answer: A concise expression implementing a functional interface's single abstract method.

Q: What is a Stream?

Answer: A pipeline abstraction for processing elements with operations such as filter, map and collect.

Q: What is dependency injection?

Answer: Supplying an object's dependencies from outside rather than having the object construct them itself.

Q: Why constructor injection?

Answer: Dependencies are explicit, can be final, and the object can be created in a valid state; testing is straightforward.

Q: What is REST?

Answer: An architectural style for resource-oriented APIs commonly using HTTP methods and representations such as JSON.

Q: 401 vs 403?

Answer: 401 indicates missing/invalid authentication; 403 indicates the request is understood/authenticated but access is not allowed.

Q: JPA vs Hibernate?

Answer: JPA is a persistence specification; Hibernate is a popular JPA implementation.

Q: Lazy vs eager?

Answer: Lazy defers loading until needed; eager loads earlier. Lazy is often safer for large relationships but must be managed carefully.

Q: What is N+1?

Answer: One query loads parent rows and additional queries are issued per parent for related data, causing excessive database calls.

Q: Authentication vs authorization?

Answer: Authentication verifies identity; authorization determines permissions.

Q: What is JWT?

Answer: A signed token format commonly used to carry claims between parties; validate signature and relevant claims.

Q: What is a microservice?

Answer: An independently deployable service organized around a cohesive business capability.

Q: What is a circuit breaker?

Answer: A resilience pattern that stops calls to an unhealthy dependency temporarily to prevent cascading failure.

Q: Why RabbitMQ?

Answer: To decouple components and support asynchronous, buffered, event-driven or task-based processing.

Q: What is a database index?

Answer: An auxiliary data structure that can speed suitable reads at the cost of storage and write overhead.

Rapid-fire checklist

- Explain one project end-to-end without reading notes.
- Write a CRUD REST API from memory.
- Explain controller → service → repository.
- Write a global exception handler.
- Explain @OneToMany / @ManyToOne and mappedBy.
- Write a JOIN and GROUP BY query.
- Explain JWT authentication flow.
- Explain one microservice failure scenario and a circuit breaker.
- Explain producer → exchange → queue → consumer in RabbitMQ.
- Give time complexity for your common array/string solutions.

19. Practical Project Blueprint

Build one small project that demonstrates the stack. A simple Employee / Task Management API is enough for an intern interview.

Suggested stack

- Java 17 or Java 21

- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- MySQL/PostgreSQL
- Bean Validation
- Spring Security + JWT
- RabbitMQ for one asynchronous event

Architecture

```

Controller
↓
Service
↓
Repository
↓
JPA/Hibernate
↓
Database

```

Optional:

Service → publish event → RabbitMQ → Consumer

Features to implement

- Create/read/update/delete users or tasks.
- Validation for request DTOs.
- Global exception handling.
- Search/filter endpoint using query parameters.
- One relationship, e.g. Department → Employees.
- JWT login + protected endpoints.
- Publish TaskCreated event to RabbitMQ.
- SQL query for a report such as department-wise task count.

How to explain the project in an interview

1. Problem:
"I built an API to manage ..."
2. Architecture:
"Controller handles HTTP, service contains business rules, repository handles persistence."
3. Database:
"I used JPA/Hibernate and mapped ..."
4. Security:
"Login authenticates the user and returns a JWT.
Protected endpoints validate the token."
5. Messaging:
"When X happens, the service publishes an event to RabbitMQ."
6. Error handling:
"REST controllers use a global exception handler."
7. Challenge:
Explain one bug/performance issue you solved.
8. Improvement:
Explain what you would add next.

Final preparation strategy

Phase 1: Java + OOP + Collections + Exceptions

Phase 2: Java 8 + SQL

Phase 3: Spring Boot + REST + JPA/Hibernate

Phase 4: Security + Microservices + RabbitMQ

Phase 5: Build and explain one project + mock interview

Reference Notes

The Java 9–21 feature comparison in these notes was checked against Oracle's Java language-change documentation. [\[cite\turn0search0\turn0search1\]](#)

Spring Security behavior was checked against Spring Boot's current reference documentation. [\[cite\turn0search9\]](#)

RabbitMQ concepts and Spring AMQP examples were checked against the official RabbitMQ tutorials, including Hello World, Work Queues, Routing, Publish/Subscribe and Topics.

[\[cite\turn0search7\turn0search8\turn0search10\turn0search3\turn0search4\]](#)

These notes are intentionally interview-oriented. They simplify some implementation details; for production systems, consult the official documentation for the exact framework/JDK/database version you are using.